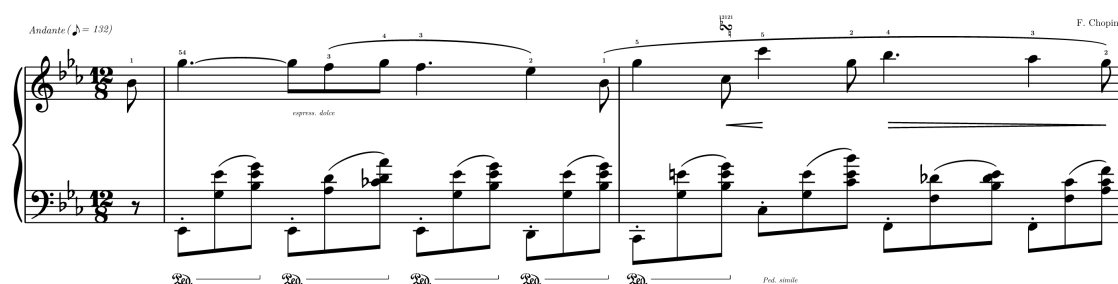


typed-scores

Western music notation, natively in Typst

User Guide



Version 0.4.0

Contents

1	Introduction	2
2	Getting started	3
2.1	Installation and import	3
2.2	Quick measure with <code>bar()</code>	3
2.3	Light and dark notation	3
3	Scores and systems	5
3.1	One staff	5
3.2	Multiple staves	5
3.3	Independent voices	6
3.4	Bar metadata	8
3.5	Harmony symbols	9
3.6	Lyrics	10
3.7	Barlines, navigation, and labeled endings	11
3.8	Score arguments	14
3.9	Indentation and justification	15
4	Event language	17
4.1	Duration inheritance	17
4.2	Pitch spelling and relative octaves	17
4.3	Relative chords	17
4.4	Beam boundaries	18
4.5	Tuplets	18
4.6	Grace notes	19
4.7	Tremolos and arpeggios	20
4.8	Expressive annotations	20
5	Validation and layout	22
5.1	Worked example: Chopin	22
6	Release showcase	24
7	Limitations	26
7.1	Current scope	26
7.2	Grammar at a glance	26
7.3	License and credits	26

1 Introduction

typed-scores turns compact musical text into engraved western notation inside Typst. A Rust/WASM plugin parses pitches, durations, exact onsets, and beam groups; Typst and CeTZ lay out the score with bundled Bravura glyphs.

The package is designed for lecture notes, theory worksheets, analytical examples, short compositions, and reusable excerpts whose musical content should remain readable and versionable as text.

Note. The public API intentionally has only two entry points: `score` for complete scores and `bar` for a quick one-staff measure.

```
1 #score(  
2   time: "4/4",  
3   bars: ((notes: "c4:q d e f"),),  
4 )
```

 Typst



2 Getting started

2.1 Installation and import

Import the package from the Typst preview namespace:

```
1 #import "@preview/typed-scores:0.4.0": *
```

 Typst

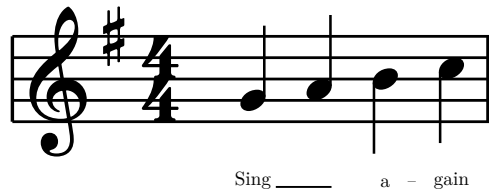
The examples in this guide use lowercase pitch letters for fast entry. Uppercase remains accepted with identical pitch and octave semantics.

2.2 Quick measure with `bar()`

`bar` is a convenience wrapper over the same score pipeline used by `score`. It accepts a note string, optional lyrics, and its clef, key, and optional meter.

```
1 #bar(  
2   "g4:q a b c5",  
3   lyrics: "Sing __ a -- gain",  
4   clef: "treble",  
5   key: "G",  
6   time: "4/4",  
7 )
```

 Typst



With `time` omitted, the `bar` remains useful for a short fragment and does not perform full-measure duration validation.

Argument	Type	Default	Description
<code>notes</code>	<code>str</code>	required	One event string.
<code>lyrics</code>	<code>str</code> <code>array</code> <code>none</code>	<code>none</code>	One lyric verse or an array of verses.
<code>clef</code>	<code>str</code>	<code>"treble"</code>	Treble, bass, alto, or tenor clef.
<code>key</code>	<code>str</code>	<code>"C"</code>	Major or minor key signature.
<code>time</code>	<code>str</code> <code>none</code>	<code>none</code>	Meter and expected duration, such as <code>"4/4"</code> .
<code>theme</code>	<code>str</code>	<code>"auto"</code>	Notation ink: <code>light</code> uses black, <code>dark</code> uses white, and <code>auto</code> follows the surrounding solid text fill.
<code>lyric-size</code>	<code>number</code>	<code>0.9</code>	Lyric text size in staff spaces.
<code>lyric-font</code>	<code>str</code> <code>none</code>	<code>none</code>	Optional font family for lyrics.
<code>lyric-gap</code>	<code>number</code>	<code>0.8</code>	Clearance above the first verse.
<code>verse-gap</code>	<code>number</code>	<code>1.45</code>	Baseline distance between verses; must be at least <code>lyric-size</code> .

2.3 Light and dark notation

Use `theme: "dark"` when placing a score on a dark surface. It recolors staff geometry, text, and every bundled Bravura glyph together. The default `theme: "auto"` follows the surrounding Typst text fill; it cannot inspect CSS changes applied by an HTML host after Typst has exported the SVG.

```
1 #block(fill: rgb("#202124"),  
  inset: 10pt)[  
2   #bar(  
3     "c4:q d e f",  
4     time: "4/4",  
5     theme: "dark",  
6   )  
7 ]
```

 Typst

3 Scores and systems

Every score is an ordered `bars`: list. Each bar is a dictionary: it contains staff event strings and optional metadata that belongs to that musical bar. Never place structural bar delimiters inside an event string.

3.1 One staff

For one staff, omit `staves`: and use the reserved `notes`: field in every bar. There is no artificial staff name to repeat. A string is one rhythmic voice; an array of two to four strings creates independent simultaneous voices on the same staff.

```
1 #score(  
2   clef: "treble",  
3   time: "4/4",  
4   bars: (  
5     (notes: "c5:q d5:q e5:q f5:q"),  
6     (notes: "g5:h e5:h"),  
7   ),  
8 )
```

 Typst

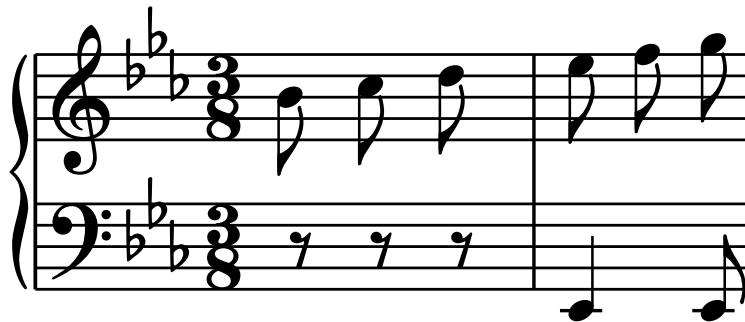
3.2 Multiple staves

For two or more staves, declare a map of permanent staff IDs and clefs. Every bar must then contain exactly one string or voice-string array for each declared ID.

```

1  #score(
2    staves: (
3      upper: (clef: "treble"),
4      lower: (clef: "bass"),
5    ),
6    key: "Eb",
7    time: "3/8",
8    bars: (
9      (
10       upper: "bb4:e c5:e d5:e",
11       lower: "r:e r:e r:e",
12     ),
13     (
14       upper: "eb5:e f5:e g5:e",
15       lower: "eb2:q eb2:e",
16     ),
17   ),
18   beams: true,
19 )

```

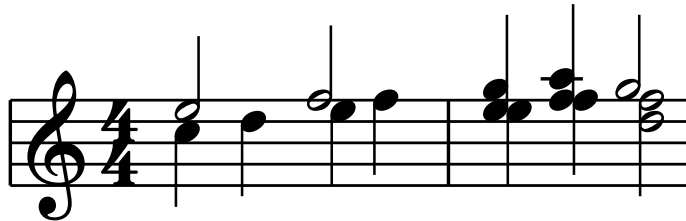
 Typst


Staff IDs are Typst dictionary keys, so keep them short and machine-oriented. They cannot reuse reserved bar fields such as `notes`, `lyrics`, `key`, `time`, `partial`, `tempo`, or `harmony`.

3.3 Independent voices

Use an array when notes on one staff have independent rhythms. The voice count must stay constant for that staff throughout the score. Following LilyPond's standard polyphonic convention, the first voice uses upward stems, the second downward stems, and later voices alternate. Rests move into separate vertical lanes. Coincident identical noteheads merge; unisons and seconds that cannot merge are shifted horizontally while retaining their shared onset.

```
1  #score(  
2    time: "4/4",  
3    bars: (  
4      (notes: (  
5        "e5:h f5:h",  
6        "c5:q d5:q e5:q f5:q",  
7      )),  
8      (notes: (  
9        "(e5 g5):q (f5 a5):q g5:h",  
10       "e5:q f5:q (d5 f5):h",  
11     )),  
12  ),  
13 )
```

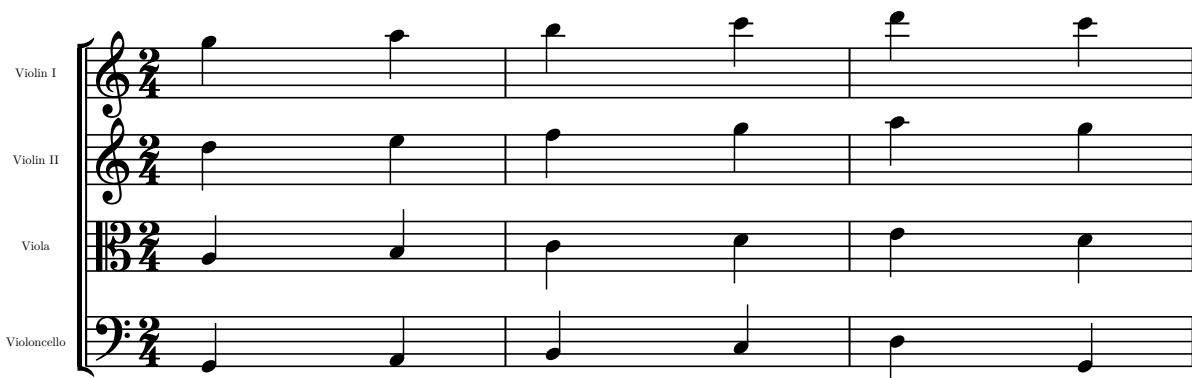
 Typst

Each staff may also define `label` for its first-system name and `short-label` for the optional name on later systems. This follows LilyPond's `instrumentName` / `shortInstrumentName` behavior: names live in a shared column to the left of the group, center horizontally within that column, and center vertically on their individual staff. The compact example below shows the full first-system names; wrapped systems substitute the short names.

```

1  #score(
2    staves: (
3      violin-one: (clef: "treble", label: "Violin I", short-label: "Vln. I"),
4      violin-two: (clef: "treble", label: "Violin II", short-label: "Vln. II"),
5      viola: (clef: "alto", label: "Viola", short-label: "Vla."),
6      cello: (clef: "bass", label: "Violoncello", short-label: "Vc."),
7    ),
8    group: "bracket",
9    time: "2/4",
10   scale: 0.58,
11   ragged-right: false,
12   bars: (
13     (violin-one: "g5:q a5:q", violin-two: "d5:q e5:q", viola: "a3:q b3:q", cello:
14       "g2:q a2:q"),
15     (violin-one: "b5:q c6:q", violin-two: "f5:q g5:q", viola: "c4:q d4:q", cello:
16       "b2:q c3:q"),
17     (violin-one: "d6:q c6:q", violin-two: "a5:q g5:q", viola: "e4:q d4:q", cello:
18       "d3:q g2:q"),
19   ),
20 )

```

 Typst


The `group` argument controls the symbol at the left of a multi-staff system: `"brace"`, `"bracket"`, `"line"`, or `"none"`. Its default, `auto`, uses no symbol for one staff, a brace for two staves, and a bracket for larger ensembles. Brace groups connect barlines across their staves; the other styles keep each staff's barline separate.

3.4 Bar metadata

`clef`, `key`, `time`, and `tempo` may be set on any bar and persist until changed. `partial` changes duration validation only: it does not change the displayed meter. When a key change removes old alterations, cancellation naturals are engraved before the replacement key signature. This replaces separate section or parallel-array forms.

For a single staff, a clef change is `clef: "bass"`. For multiple staves, use a dictionary containing only the staves that change, such as `clef: (left: "bass")`. As in LilyPond, a mid-system change is reduced in size; the active clef is printed full-size when a new system begins.

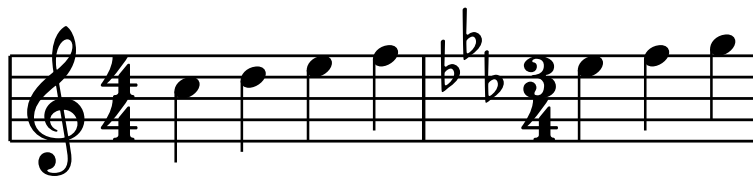
```

1  #score(
2      clef: "treble",
3      key: "C",
4      time: "4/4",
5      bars: (
6          (
7              notes: "c5:q d5:q e5:q f5:q",
8              tempo: (text: [Moderato], beat: "quarter", bpm: 108),
9          ),
10         (
11             key: "Eb",
12             time: "3/4",
13             notes: "eb5:q f5:q g5:q",
14         ),
15     ),
16 )

```

 Typst

Moderato (♩ = 108)



For a metronome mark, use a tempo dictionary rather than inserting a Unicode note character. `beat` accepts `whole`, `half`, `quarter`, `eighth`, `sixteenth`, or `thirty-second`, and the matching note glyph is drawn automatically. `text` is optional, so `(beat: "eighth", bpm: 132)` produces just the note-and-number mark.

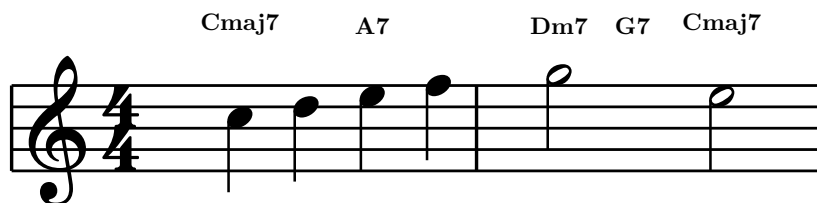
3.5 Harmony symbols

Use a bar's `harmony` field for chord symbols above the top staff. It is an independent duration-bearing sequence: each token is author-controlled text followed by `:` and a duration, and the sequence must fill the active bar. Harmony changes use the same onset positions as the notes, including changes that fall between melody note onsets. Each symbol is centered on the onset where the harmony becomes active, independent of its duration. Symbols are text, so jazz spellings such as `Cmaj7`, `F#7(b9)`, `Bb/D`, and `N.C.` remain under the author's control.

```

1  #score(
2    time: "4/4",
3    bars: (
4      (
5        notes: "c5:q d5:q e5:q f5:q",
6        harmony: "Cmaj7:h A7:h",
7      ),
8      (
9        notes: "g5:h e5:h",
10       harmony: "Dm7:q G7:q Cmaj7:h",
11      ),
12    ),
13  )

```

 Typst


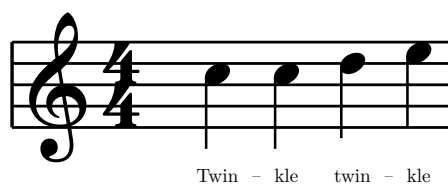
3.6 Lyrics

Use a bar's `lyrics` field to align syllables with the main pitched events of a staff's first voice. A plain string creates one verse. `--` draws a hyphen without consuming an event, `_` consumes one event and extends the preceding syllable, and `_` consumes one event without printing text. Written rests and grace notes do not consume lyric tokens. Every verse present in a bar must account for all of that bar's main pitched events, which keeps alignment mistakes visible instead of silently shifting later words.

```

1  #score(
2    time: "4/4",
3    bars: (
4      (
5        notes: "c5:q c d e",
6        lyrics: "Twin -- kle twin -- kle",
7      ),
8    ),
9  )

```

 Typst


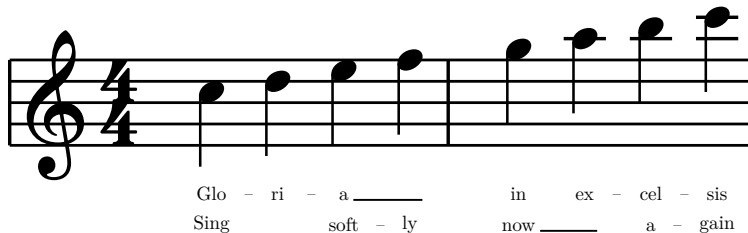
An array creates multiple verses. Use an underscore inside a syllable token to print a space, while a standalone underscore remains a skip. Hyphens and melisma extenders may cross barlines and resume when the score wraps.

```

1  #score(
2    time: "4/4",
3    width: 34,
4    bars: (
5      (
6        notes: "c5:q d e f",
7        lyrics: ("Glo -- ri -- a __", "Sing _ soft -- ly"),
8      ),
9      (
10       notes: "g5:q a b c6",
11       lyrics: ("in ex -- cel -- sis", "now __ a -- gain"),
12     ),
13   ),
14 )

```

t Typst



In a declared multi-staff score, `lyrics` is a dictionary whose keys are staff IDs. Each value is one verse string or an array of verses. Omit staves that have no lyrics in that bar. Lyrics always follow the first voice of their target staff; use that voice for the sung line when a staff is polyphonic.

```

1  lyrics: (
2    soprano: ("Hal -- le -- lu -- jah", "Praise _ the Lord"),
3    bass: "Low __",
4  )

```

t Typst

Lyric widths constrain the shared onset grid before measures are packed, so long syllables can widen bars and change line breaks. Verse lanes are also part of the staff's vertical ink extent: they clear dynamics and other below-staff marks, enlarge automatic multi-staff gaps, and contribute to system bounds. Use `lyric-size`, `lyric-font`, `lyric-gap`, and `verse-gap` to tune their typography and vertical rhythm.

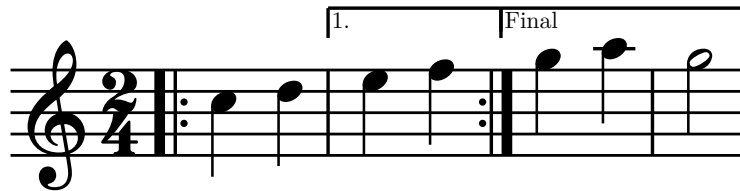
3.7 Barlines, navigation, and labeled endings

Repeat signs belong to a bar edge. Use `repeat-start` on a left edge and `repeat-end` on a right edge. A labeled ending is a span: mark its first bar with `start: true` and its last bar with `stop: true`. `label` is rendered literally, so punctuation and text such as "1." or "Final" are both valid.

```

1  #score(
2    time: "2/4",
3    bars: (
4      (barline: (left: "repeat-start"), notes: "c5:q d5:q"),
5      (
6        barline: (right: "repeat-end"),
7        ending: (label: "1.", start: true, stop: true),
8        notes: "e5:q f5:q",
9      ),
10     (ending: (label: "Final", start: true), notes: "g5:q a5:q"),
11     (ending: (label: "Final", stop: true), notes: "g5:h"),
12   ),
13 )

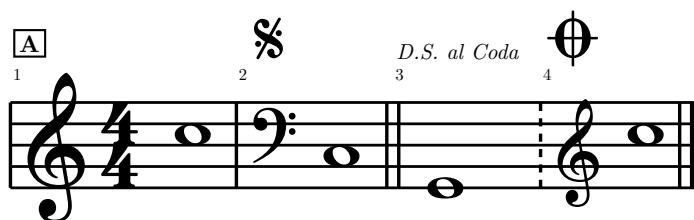
```

 Typst


A repeat end followed immediately by a repeat start becomes one double-sided repeat barline. Volta brackets continue across system wraps. Endings are currently sequential: one bracket must stop before another one starts.

A right bar edge may also be **double**, **final**, or **dashed**. Use a bar's `rehearsal` field for a boxed rehearsal label and `navigation` for "segno", "coda", arbitrary text, or Typst content. Marks attach to the beginning of their bar. When a bar number and mark coincide, the number stays closest to the staff and the mark stacks above it. Long boundary text reserves enough horizontal space to clear the following bar. `bar-numbers: "systems"` prints the first bar number of later systems, matching LilyPond's usual sparse numbering; `"all"` prints every bar number.

```
1  #score(  
2    time: "4/4",  
3    bar-numbers: "all",  
4    bars: (  
5      (rehearsal: "A", notes: "c5:w"),  
6      (clef: "bass", navigation: "segno", barline: (right: "double"), notes: "c3:w"),  
7      (navigation: "D.S. al Coda", barline: (right: "dashed"), notes: "g2:w"),  
8      (clef: "treble", navigation: "coda", barline: (right: "final"), notes: "c5:w"),  
9    ),  
10  )
```

 Typst

3.8 Score arguments

Argument	Type	Default	Description
<code>clef</code>	<code>str</code>	<code>"treble"</code>	Clef for the implicit single staff only.
<code>staves</code>	<code>dictionary</code> <code>none</code>	<code>none</code>	Permanent staff-ID map. Each spec has <code>clef</code> , plus optional first-system <code>label</code> and later-system <code>short-label</code> .
<code>bars</code>	<code>array</code>	<code>()</code>	Required non-empty array of bar dictionaries.
<code>key</code>	<code>str</code>	<code>"C"</code>	Initial key signature.
<code>time</code>	<code>str</code>	<code>"4/4"</code>	Initial meter and full-bar duration.
<code>theme</code>	<code>str</code>	<code>"auto"</code>	Notation ink: <code>light</code> uses black, <code>dark</code> uses white, and <code>auto</code> follows the surrounding solid text fill.
<code>tempo</code>	<code>str</code> <code>content</code> <code>dictionary</code> <code>none</code>	<code>none</code>	Tempo text, or <code>(text:, beat:, bpm:)</code> for an engraved metronome mark.
<code>composer</code>	<code>str</code> <code>content</code> <code>none</code>	<code>none</code>	Composer credit above the first system.
<code>width</code>	<code>number</code> <code>none</code>	<code>none</code>	Packing width in staff spaces.
<code>scale</code>	<code>number</code>	<code>1.0</code>	Uniform staff-space scale.
<code>note-spacing</code>	<code>number</code>	<code>3.1</code>	Horizontal density: the width in staff spaces of a quarter note in a bar of quarters; other durations scale logarithmically from the bar's shortest note.
<code>lyric-size</code>	<code>number</code>	<code>0.9</code>	Lyric text size in staff spaces.
<code>lyric-font</code>	<code>str</code> <code>none</code>	<code>none</code>	Optional lyric font family.
<code>lyric-gap</code>	<code>number</code>	<code>0.8</code>	Clearance between existing below-staff ink and the first verse.
<code>verse-gap</code>	<code>number</code>	<code>1.45</code>	Baseline distance between lyric verses; must be at least <code>lyric-size</code> .
<code>beams</code>	<code>bool</code>	<code>false</code>	Connect automatic beam groups.
<code>staff-gap</code>	<code>number</code> <code>none</code>	<code>none</code>	Fixed distance between adjacent staff lines.
<code>group</code>	<code>auto</code> <code>str</code>	<code>auto</code>	Auto, brace, bracket, line, or none.
<code>wrap</code>	<code>bool</code>	<code>true</code>	Wrap complete bars into systems.
<code>indent</code>	<code>number</code>	<code>0</code>	First-system indentation in staff spaces. The system still ends at the normal right edge.
<code>short-indent</code>	<code>number</code>	<code>0</code>	Indentation for every system after the first.

<code>ragged-right</code>	<code>auto bool</code>	<code>auto</code>	Leave every system at natural width. Auto is ragged only for a one-system score.
<code>ragged-last</code>	<code>bool</code>	<code>false</code>	Leave only the final system at its natural width.
<code>system-gap</code>	<code>length</code>	<code>1.2em</code>	Vertical gap between systems.
<code>bar-numbers</code>	<code>false str</code>	<code>false</code>	Use <code>systems</code> for later system starts or <code>all</code> for every bar.
<code>first-bar-number</code>	<code>int</code>	<code>1</code>	Positive number assigned to the first bar.

3.9 Indentation and justification

Multi-system scores fill `width` by default: note spacing stretches or compresses so each system's bars fill the line, and adjacent systems keep a similar density rather than alternating cramped and loose lines. There is no fixed bar count per line. A one-system score stays at natural width with the default `ragged-right: auto`.

Use `indent` for the first system and `short-indent` for every later one; both reduce the usable line width while retaining a shared right edge. Set `ragged-last: true` for a natural-width final line.

```
1  #score(  
2    time: "4/4",  
3    width: 36,  
4    indent: 3.5,  
5    ragged-last: true,  
6    bars: (  
7      (notes: "c5:q d e f"),  
8      (notes: "g5:q a b c6"),  
9      (notes: "c6:h g5:h"),  
10     (notes: "f5:q e d c"),  
11     (notes: "b4:h c5:h"),  
12     (notes: "d5:q e f g"),  
13   ),  
14 )
```

 Typst

4 Event language

Syntax	Meaning
<code>c4:q</code> or <code>c4q</code>	Quarter note.
<code>ce</code> or <code>c:e</code>	Relative C with an eighth-note duration.
<code>c4:q d e f</code>	Four quarter notes using inherited register and duration.
<code>bb4:e.</code>	Dotted eighth note.
<code>c##5:q</code> / <code>dbb5:q</code>	Double-sharp / double-flat quarter note.
<code>(a4 c e):h (f a c)</code>	Relative chord pitches and inherited duration.
<code>r:q</code>	Quarter rest.
<code>_</code>	Automatic rest placeholder.
<code>~</code>	Tie the previous event to the next event.
<code>/</code>	Break the automatic beam before the next event.
<code>-</code>	Join adjacent flagged events into one beam group.
<code>tuplet 3:2 { c:e d e }</code>	Inline time-scaled music group.
<code>acciaccatura { d:e } f:q</code>	Slashed single grace note before F.
<code>tremolo 16 { c:h g:h }</code>	Two-note alternating sixteenth tremolo.

4.1 Duration inheritance

Durations are `w`, `h`, `q`, `e`, `s`, and `t`; append `.` or `..` for dots. An explicit duration becomes the current duration for that staff. Subsequent notes, chords, and ordinary rests may omit it and inherit the complete value, including dots, across barlines. With no preceding value, the default is `q`. The computed-rest placeholder `_` does not change this state, and a written colon must always be followed by a duration. An omitted duration never expands to fill the measure; normal bar-duration validation still reports an incorrect total.

4.2 Pitch spelling and relative octaves

Pitch letters are ASCII case-insensitive. `c4`, `C4`, and mixed-case input have identical pitch and octave semantics; this guide uses lowercase as the compact style, while uppercase remains valid. Compact `ce` and explicit `c:e` both mean an eighth-note C. Whitespace distinguishes that single event from `c e`, which means two notes using the inherited duration. Capitalization never selects an octave.

Pitches accept `#`, `b`, `##`, or `bb`. In lowercase, `bb4` is B-flat 4 and `bbb4` is B-double-flat 4. A single note with an explicit octave anchors later octave-less single notes. Each omitted octave selects the nearest diatonic pitch to the preceding resolved note, so `g4:e a b c` ends on C5 and all four notes are eighths. Accidentals and the active key signature do not affect that register choice. The anchor continues across bars independently for each staff, and any explicit octave resets it. At the beginning of an unanchored staff, treble, alto, and tenor start in octave 4; bass starts in octave 3.

4.3 Relative chords

Chords follow LilyPond's relative-entry model. The first written chord pitch is resolved from the current staff anchor. Each later chord pitch is resolved from the pitch immediately before it inside the chord. Afterward, the first written chord pitch becomes the external anchor for the next event. For example,

(a4 c e):h (f a c) resolves to (A4 C5 E5):h (F4 A4 C5):h. Explicit octaves remain exact overrides for open or otherwise non-nearest voicings.

4.4 Beam boundaries

Standalone / and - are local beam-boundary controls. / splits before the next event; - joins the flagged event on each side even across an automatic metric boundary. A join at an edge, beside a rest, or beside an unflagged duration is an error. For example, this 3/8 measure is grouped 3+3 rather than the automatic 2+2+2 sixteenths, and the next bar demonstrates relative chord voicing. No **key** argument is needed here: **score** defaults to C major.

```

1  #score(
2    time: "3/8",
3    beams: true,
4    bars: (
5      (notes: "g4:s a - b / c - d e"),
6      (notes: "(c5 e g):e (d f a) (e g b)"),
7    ),
8  )

```

 Typst



4.5 Tuplets

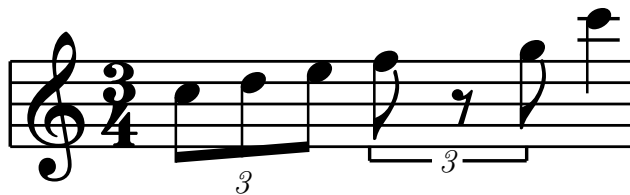
Tuplets are inline music groups: `tuplet 3:2 { c:e d e }` writes three eighths in the time normally occupied by two. Any positive ratio is supported, and groups may nest. The group preserves pitch and duration inheritance, so only its first event needs an explicit octave or duration when the surrounding context already supplies one.

By default the numerator is centered on the group, and a bracket is omitted when one visible beam spans every event in the tuplet; otherwise a bracket is drawn. Written rests keep the bracket. Use optional group controls when the engraving needs an explicit choice: **bracket=always**, **bracket=never**, **side=above**, or **side=below**.

```

1 #score(
2   time: "3/4",
3   beams: true,
4   bars: (
5     (
6       notes: "tuplet 3:2 { c5:e d e } tuplet 3:2 { f5:e r:e g5:e } c6:q",
7     ),
8   ),
9 )

```

 Typst


4.6 Grace notes

Grace groups stay inline and consume no bar time. `grace { ... }` writes plain small grace notes. `acciaccatura { ... }` adds a slur to the following principal note, as does `appoggiatura { ... }`. For a multi-note group, the slur starts at the first grace event. A single flagged acciaccatura also receives the conventional slash; a multi-note beamed acciaccatura does not. Consecutive flagged grace notes beam as an independent group. Their written durations control flags and beams but do not contribute to the measure total.

```

1 #score(
2   time: "4/4",
3   beams: true,
4   bars: (
5     (notes: "acciaccatura { b4:e } c5:q acciaccatura { d5:s e } f5:q appoggiatura
6       { g5:e } a5:q grace { b5:s c6 } d6:q"),
7   ),
8 )

```

 Typst


A grace group must contain notes or chords and must precede a principal event. Rests, automatic rests, triplets, alternating tremolos, and nested grace groups are rejected inside it so timing remains unambiguous.

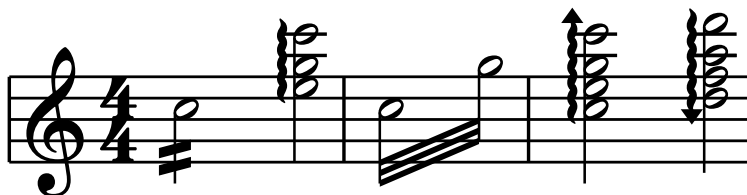
4.7 Tremolos and arpeggios

For a repeated single event, append `tremolo=8`, `16`, `32`, or `64` in its annotation block. These values draw one, two, three, or four strokes respectively, independent of the event's written duration. For an alternation, place exactly two equal-duration notes or chords in `tremolo subdivision { ... }`; the connecting strokes replace ordinary flags or beams.

Append `arpeggio`, `arpeggio=up`, or `arpeggio=down` to a chord. The arpeggio wave spans the full chord immediately to its left; the directional forms add an arrow at the appropriate end.

```
1 #score(
2   time: "4/4",
3   bars: (
4     (notes: "c5:h[tremolo=16] (e5 g5 c6):h[arpeggio]"),
5     (notes: "tremolo 16 { c5:h g5:h }"),
6     (notes: "(c5 e5 g5 c6):h[arpeggio=up] (d5 f5 a5 d6):h[arpeggio=down]"),
7   ),
8 )
```

t Typst



4.8 Expressive annotations

Annotations follow an event in square brackets. Supported annotations include fingering (`f=4`), staccato, staccatissimo, tenuto, accent, marcato, text directions, turns, fermatas (`fermata`), breath marks (`breath`), named slurs, pedal spans, hairpins, and dynamics (`dyn=p`, `dyn=pp`, `dyn=mf`, `dyn=sfz`, and the other standard SMuFL combinations).

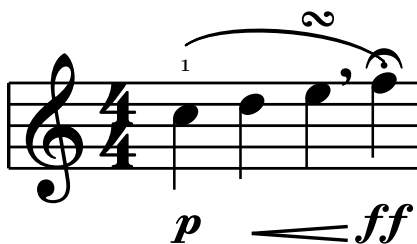
Dynamics, hairpins, slurs, articulations, and fingerings avoid collisions with each other automatically: a hairpin shortens around an adjacent dynamic such as a closing `dyn=ff`, and fingerings rise above stacked articulations rather than overlapping them. Named slurs over beamed passages follow the beam group's stem direction, so a slur goes below when every note it spans stems upward and above when any note stems downward.

Syntax	Mark
<code>f=4</code>	Fingering.
<code>stacc</code> / <code>staccatissimo</code>	Staccato dot / wedge.
<code>tenuto</code> / <code>accent</code> / <code>marcato</code>	Articulation marks.
<code>turn</code> / <code>chromatic-turn</code>	Turn ornament.
<code>dyn=pp</code> / <code>dyn=sfz</code>	Dynamic marking.
<code>fermata</code> / <code>breath</code>	Fermata / comma breath mark.
<code>s1(... s1)</code>	Named slur.
<code>p1(... p1)</code>	Pedal span.

`h1< or h1> ... h1!`

Crescendo / diminuendo hairpin.

```
1 #score(  
2   clef: "treble",  
3   time: "4/4",  
4   beams: true,  
5   bars: (  
6     (notes: "c5:q[f=1 s1( dyn=p) d5:q[h1<] e5:q[turn breath] f5:q[s1) h1! dyn=ff  
7       fermata]"),  
8   ),  
9 )
```

 Typst

5 Validation and layout

The package validates every full bar against its active **time** signature and every pickup against **partial**. It also reports unknown staff IDs, missing staff content, malformed or undocumented annotations, impossible automatic rests, invalid slur/pedal/hairpin lifecycles, and tie errors with staff and bar context. A tie must connect immediately adjacent events with the same written pitch or chord; it is split cleanly when the score wraps and suppresses a redundant continuation accidental. Multi-staff onsets share horizontal positions, and bars remain atomic when systems wrap.

Condition	Diagnostic
Malformed pitch or duration	The parser identifies the invalid component.
Wrong full-bar duration	Actual reduced duration and expected meter.
Wrong pickup duration	Actual duration and declared partial value.
Wrong harmony duration	Actual reduced duration and expected meter.
Invalid lyric token count or control	Bar, staff, verse, expected event count, and a concrete correction.
Unknown lyric staff or dangling continuation	The invalid staff reference, hyphen, or melisma source.
Unknown or missing staff ID	The one-based bar and offending field.
Invalid span lifecycle	The slur, pedal, hairpin, or ending that cannot be paired.
Invalid tie or beam join	The adjacent events that cannot legally connect.

5.1 Worked example: Chopin

The canonical release fixture uses the multi-staff form and exercises a pickup, compound beaming, ties, slurs, dynamics, turns, pedal spans, and chromatic accidentals:

The image displays a musical score for Chopin's Andante in B-flat major, Op. 25, No. 1. The score is in 12/8 time and features a multi-staff layout with piano and right-hand parts. It includes various musical notations such as slurs, ties, dynamics (p, f), and chromatic accidentals. The score is presented in a clean, professional layout with clear notation and a multi-staff format.

The image displays two systems of musical notation for a piano piece, likely in B-flat major and 12/8 time. The notation is presented in two systems, each consisting of a right-hand (treble) and left-hand (bass) staff.

System 1: The right-hand staff begins with a piano (*p*) dynamic. It features a complex melody with many beamed sixteenth notes and slurs. The left-hand staff provides a steady bass line with chords and single notes, also featuring slurs. There are six measures in this system.

System 2: The right-hand staff begins with a forte (*f*) dynamic. The melody continues with similar complexity, including many beamed sixteenth notes and slurs. The left-hand staff continues the bass line pattern. There are six measures in this system, ending with a double bar line.

6 Release showcase

The repository’s `examples/` directory contains five reusable, source-checked fixtures and a compiled `showcase.pdf`:

- Chopin, Nocturne Op. 9 No. 2 — piano, pickup and measures 1–4;
- Mozart, Eine kleine Nachtmusik K. 525/I — four-staff strings, measures 1–4;
- Bach, Cello Suite No. 1 BWV 1007/I — solo cello, measures 1–4;
- Beethoven, the four-bar Ode to Joy theme — E-flat alto saxophone at written pitch; and
- Beethoven, Für Elise WoO 59 — piano, pickup and measures 1–4.

The saxophone fixture is an explicit transposition: its B-major notation sounds in Beethoven’s D major. The other four retain their reference score’s written pitches. This compact quartet excerpt demonstrates a bracketed ensemble:

Allegro W. A. Mozart

The single-staff case needs no staff declaration, even for a transposing instrument whose written pitch has been prepared by the author:

Allegro assai

L. van Beethoven

7 Limitations

7.1 Current scope

- One to four independent rhythmic voices per staff are supported; the count for a staff is fixed across its bars.
- Cross-staff notation is not yet in the public DSL.
- Grace groups currently exclude rests, tuplets, and nested ornamental groups.
- Arpeggio signs span one chord on one staff; cross-staff piano arpeggios are not yet supported.
- Pedals and hairpins are event-anchored and do not split automatically at a system break.
- Dense editorial markings or lyric verses may require `staff-gap`, `note-spacing`, `lyric-gap`, `verse-gap`, or `scale` adjustments.

7.2 Grammar at a glance

Text	Meaning
<code>c4:q / c4q</code>	Note with an explicit octave and duration.
<code>c4:q d e f</code>	Relative pitches inheriting the quarter duration.
<code>(a4 c e):h</code>	Relative chord.
<code>r:q / _</code>	Written rest / computed remainder rest.
<code>~</code>	Tie the preceding event.
<code>/ / -</code>	Break / force a local beam connection.
<code>tuplet 3:2 { c:e d e }</code>	Three written eighths in the time of two.
<code>acciaccatura { d:e } f:q</code>	Slashed single grace note resolving to F.
<code>tremolo 16 { c:h g:h }</code>	Two-note alternating tremolo.
<code>(c e g):h[arpeggio=up]</code>	Upward arpeggio over a chord.
<code>lyrics: "Twin -- kle _ _"</code>	Syllables with a hyphen, melisma extender, and skip.
<code>[stacc dyn=pp]</code>	One annotation block with multiple marks.
<code>barline: (left: "repeat-start")</code>	Repeat sign on a bar edge.
<code>barline: (right: "final")</code>	Final barline on the right edge.
<code>clef: "bass"</code>	Persistent single-staff clef change at a bar boundary.
<code>rehearsal: "A" / navigation: "segno"</code>	Boundary rehearsal / navigation mark.
<code>ending: (label: "Final", start: true)</code>	Literal volta label.

7.3 License and credits

Package code is released under the MIT License. Bundled Bravura glyphs are released under the SIL Open Font License 1.1. `typed-scores` uses CeTZ for drawing and Bravura's SMuFL metadata for music-glyph geometry.

The Chopin, Mozart, Bach, and Beethoven compositions used by the release fixtures are public domain. Their reusable Typst sources live in the `examples/` directory and are shared by the guide, showcase, and visual regression suite.